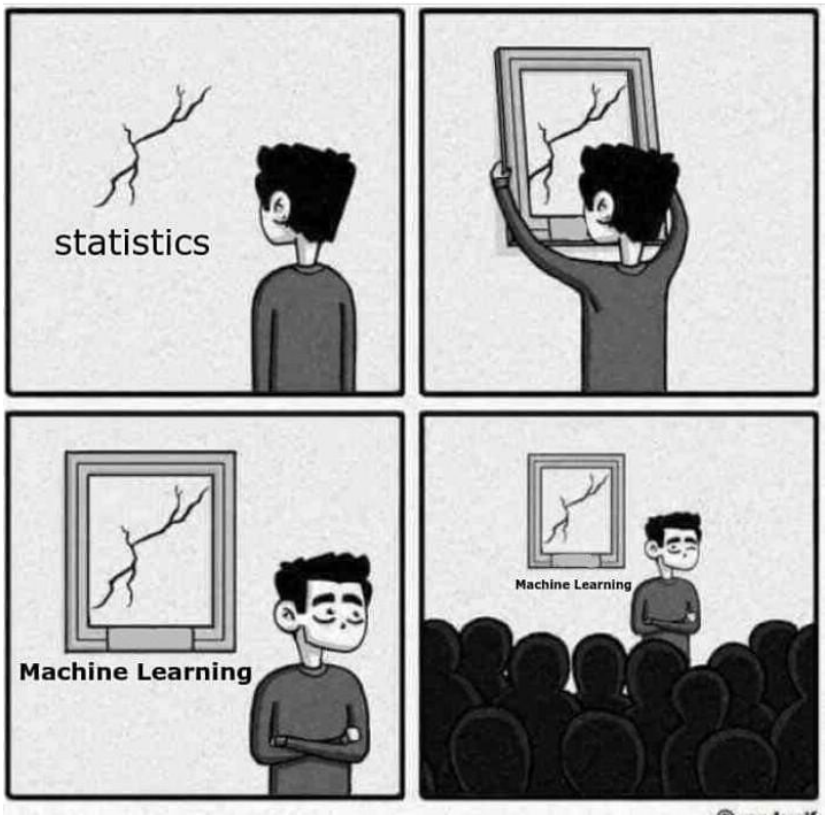
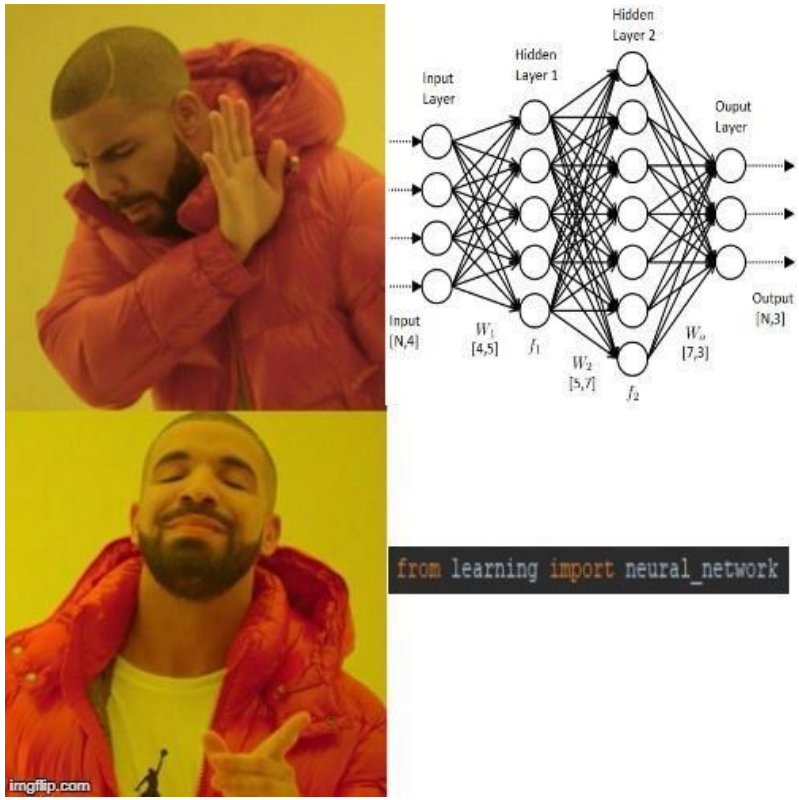
COMP S492F / 3920SEF

Artificial Neural Networks

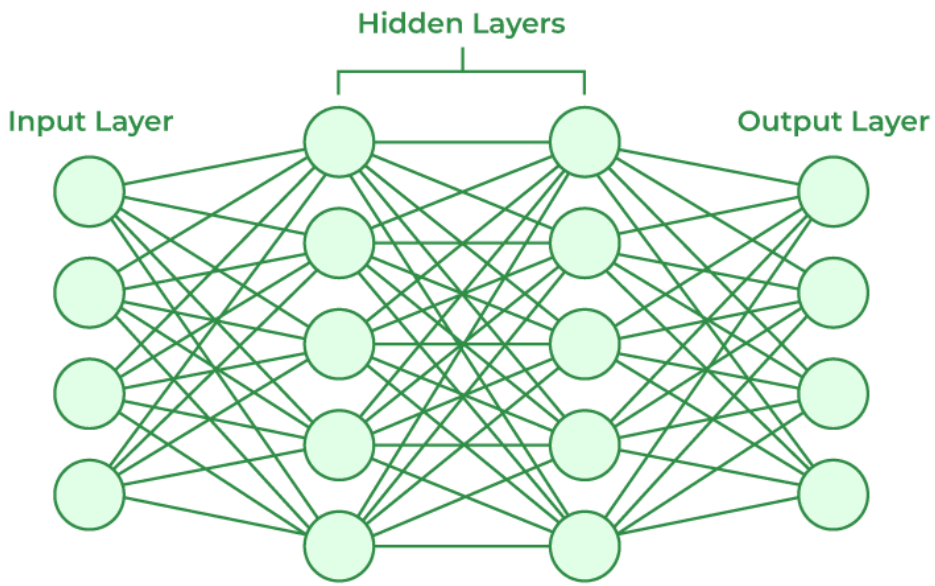




Artificial Neural Networks

Introduction to Artificial Neural Network

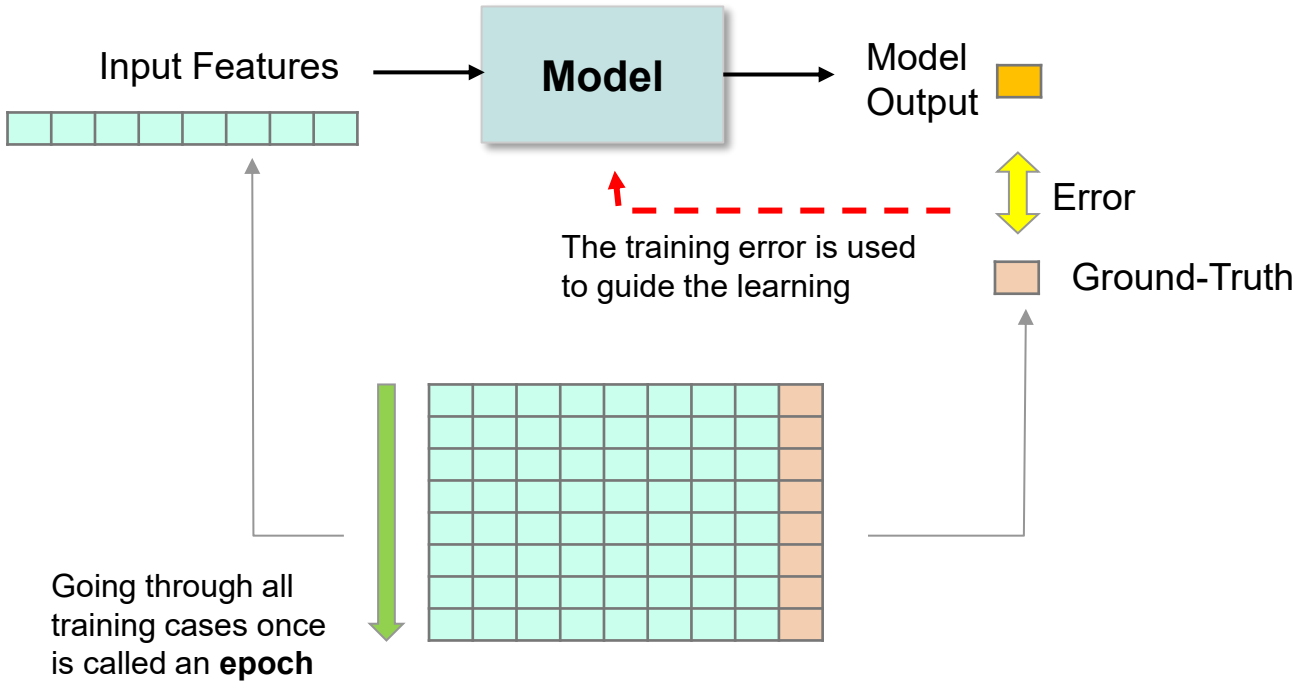
- Artificial neural networks (ANN), and in the short form neural network, is a model or a function that can be trained with data.
- ANN are made up of interconnecting neurons
  - Neurons are abstract entities that can compute an output value based on weighted input parameters.
    - The adjustment of the weights or model parameters is possible
- ANN are very powerful that it is fit training data from a complex function or model



Artificial neural network

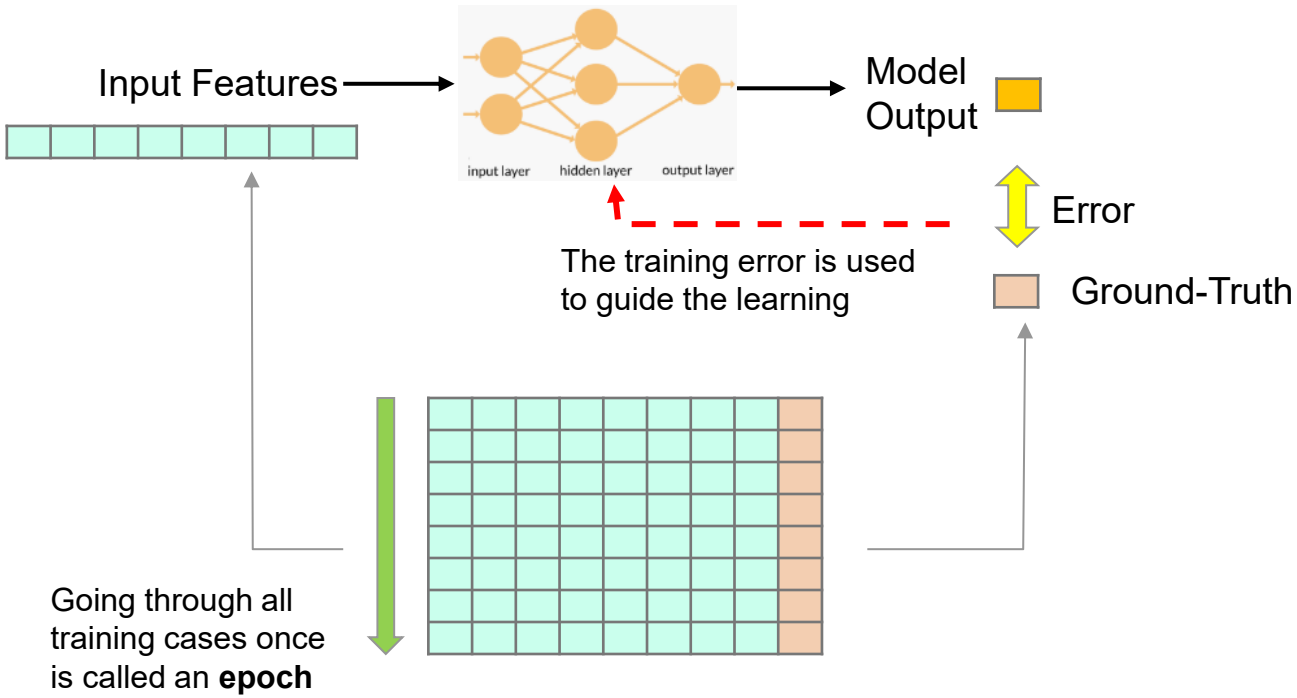
Introduction to Artificial Neural Network

What we have done in traditional parametric machine learning?



Introduction to Artificial Neural Network

In ANN, you just replace the model by a neural network

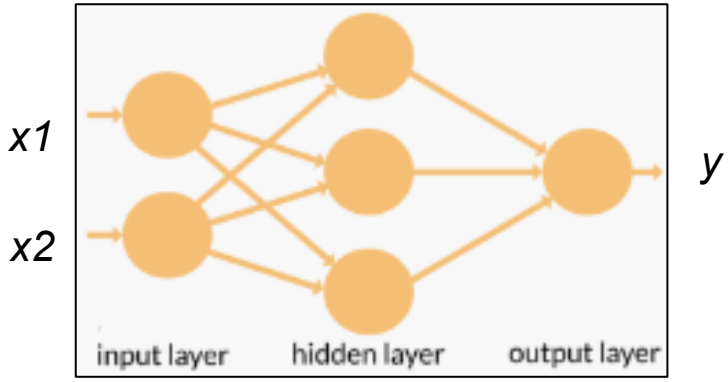


- Using ANN for machine learning is no different from other methods
  - F: X → Y
    - F is the function to be learned from dataset made up of {X, Y}
    - X are the input parameters or features (one or more)
    - Y is the target, which consists of one or more components

Structure of ANN

- The following ANN example has two input features (x1, x2) and one output variable (y)
- Internally, ANN consists of interconnecting <u>perceptrons</u>
  - There is 1 perceptron in the output layer
  - There are 3 perceptrons in the hidden layer
  - The input layer is not a perceptron (just a place for input features)

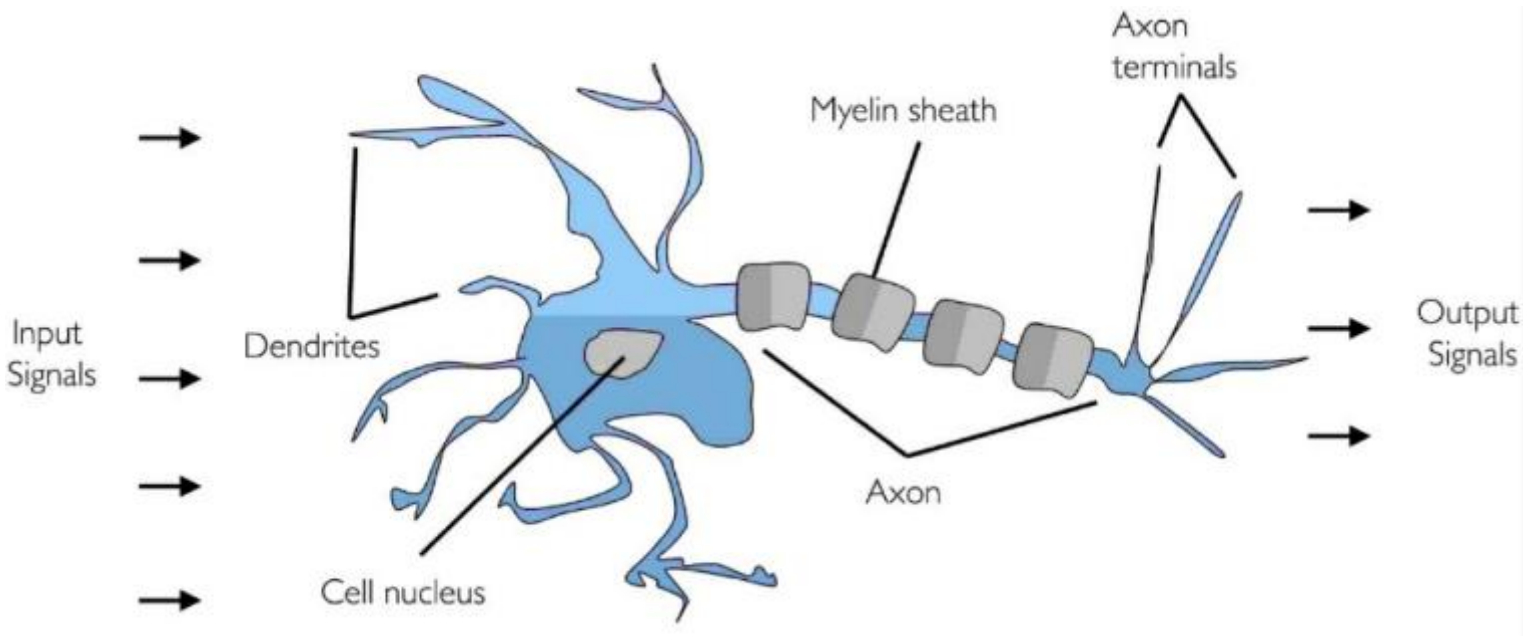
X = {x1, x2, …,, xn) → **Model** → **Y** = {y1, y2, …, ym)



What is inside the neural network?

Perceptron

- The perceptron is modelling after the neurons in human nervous systems
  - Warren McCullock and Walter Pitts published the first concept of a simplified brain cell, the so-called McCullock-Pitts (MCP) neuron, in 1943.
  - Neurons are interconnected nerve cells in the brain that are involved in the processing and transmitting of chemical and electrical signals.



Perceptron

- Simplest ANN is called a perceptron
- The output (Y) is computed from the input (X)
  - The input X have n features (in real numbers), and in the following n = 4
  - The input features are weighted sum together

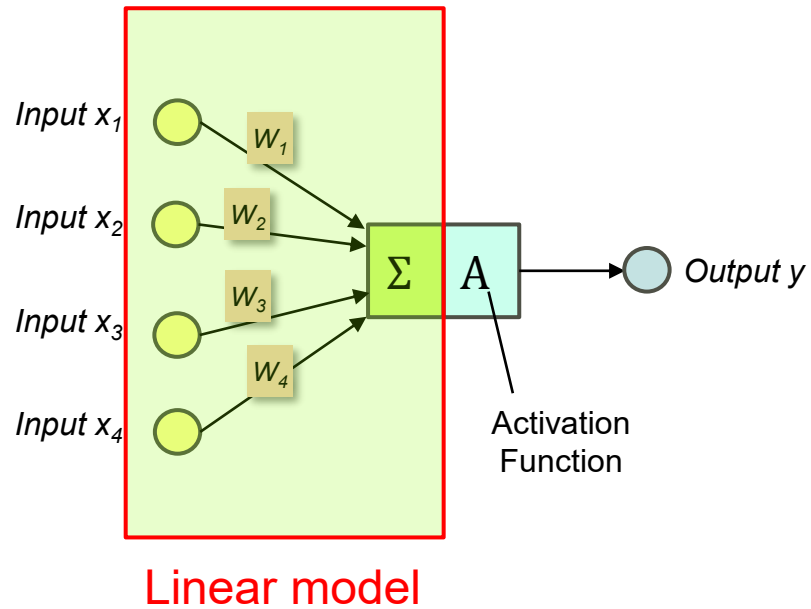
$$\sum_{j=1}^{n} x_j w_j = (x_1 w_1 + x_2 w_2 + \cdots + x_n w_n)$$

$$x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4 \; when \; n = 4$$

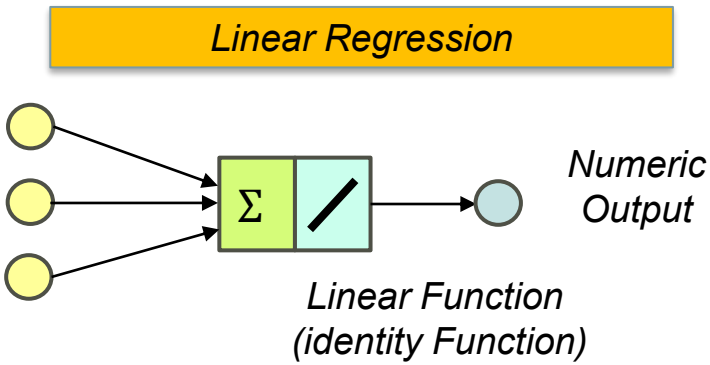
  - The weighted sum is then pass through an activation function (Act)

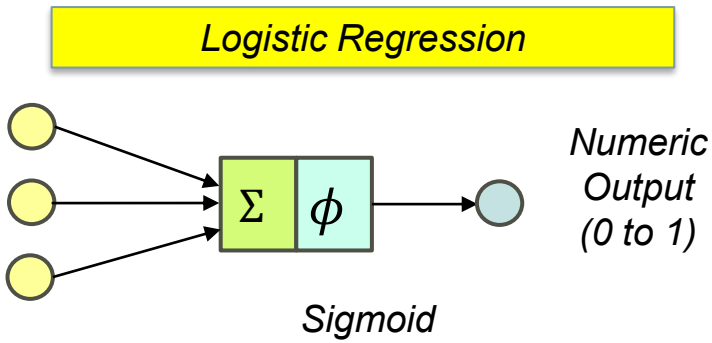
$$y = Act\left(\sum_{j=1}^{n} x_j w_j\right)$$

    - Use different activation functions for different models



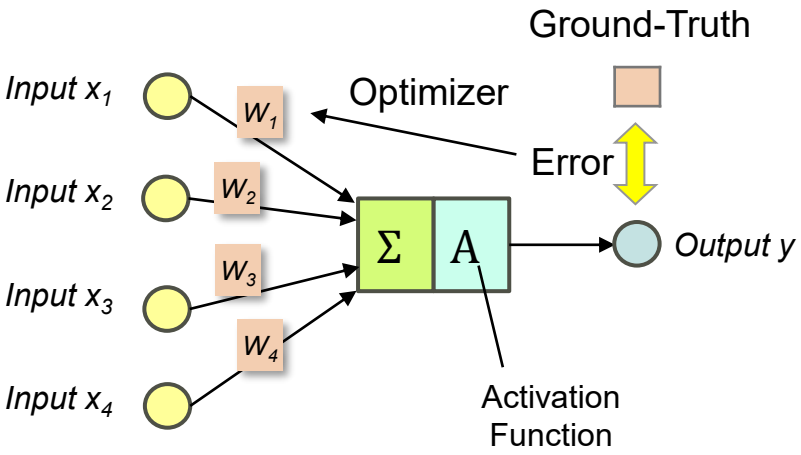
Perceptron: compared with Linear Regression / Logistic Regression





Training a Perceptron

- The learnable or trainable parameters are the weights (w1, w2, w3, w4)
  - The weights are adjusted by an optimizer algorithm
  - The error is calculated between the model output and the ground-truth
  - Adjust the weights (w1, w2, w3, w4) so that the model output is getting closer to the ground-truth



Perceptron with Bias

- Often there is an invariance part of the model called bias
  - Data representation of a measure may use different scales.
  - The following scales are virtually the same (-1 to +1) or (0 to +1) to represent unsatisfactory to satisfactory
    - Unsatisfactory (-1) to Satisfactory (+1)
    - Unsatisfactory (0) to Satisfactory (+1)
  - Similar to the parameter c in the linear regression model
- Add a bias to capture the invariance part of the model
  - The magnitude of the bias is learnable
    - One more trainable parameter

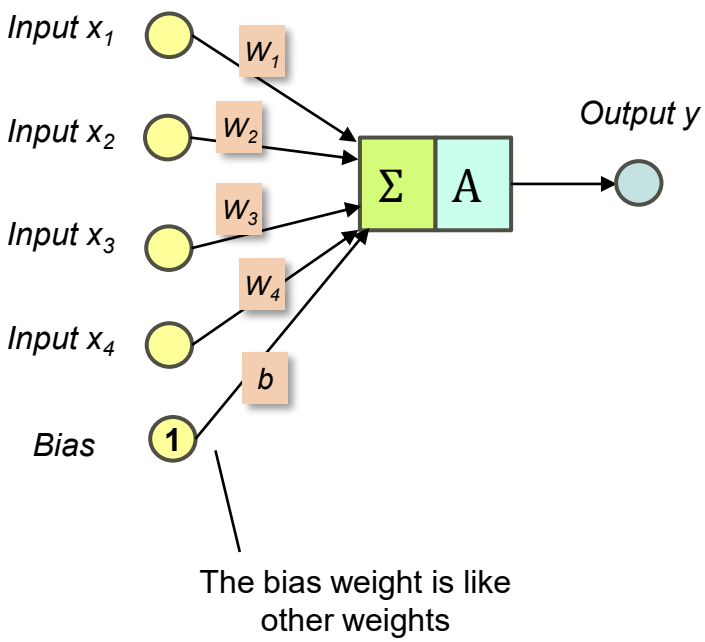
Perceptron with Bias

- The output computation is modified as below with the bias

$$y = Act\left(\sum_{j=1}^{n} x_j w_j + b\right)$$

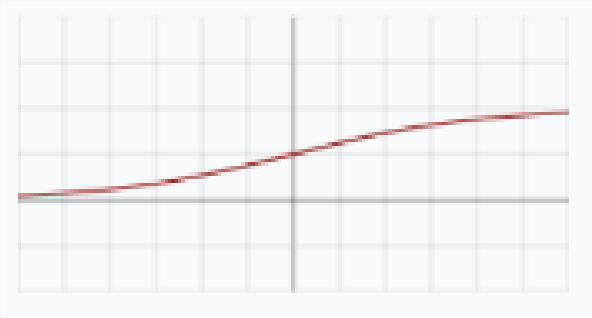
$$y = x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4 + b \; when \; n = 4$$

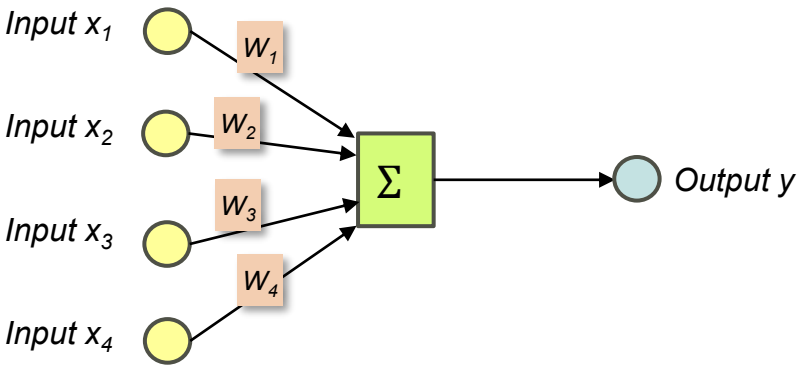
  - Note that 1 multiplied by b is just b



Activation Function

- Activation function is for regulating the output of a perceptron
  - The summation may return in a very large number
  - Overactivated
- Example: the signmod function
  - Output: 0 to 1





Activation Functions

Purpose of Activation Functions

- An activation function selected for each perceptron or neurons.
  - The input are weighted and added together
  - The input is passed to an activation function to produce output
- There are several purposes
  - Specify the output
    - The added together input can be very large or very small (negative)
    - Squash large values into small values (normalization)
    - Applying to non-bias nodes
    - Probability (0 to 1)
  - Determine whether the neurons or nodes are activated (or firing)
    - Give each node a “state” (i.e., Boolean condition)
    - A neuron is understood to be on or off (or a probability of on/off)
    - Reduce noise
  - Add “non-linearity” to the classifier
    - Real world phenomena are often non-linear
    - Two variables are not in direct proportion or direct relation

Choice of Activation Functions: Differentiable

- The Activation Function is the last step before the model output
  - This is also the first step in the updating of model parameters by an optimizer such as backpropagation
  - Any optimizer based on gradient on the error (such as gradient descent) requires the activation function to be <u>differentiable</u>
    - Differentiation is calculus used for finding the slope (that is, the gradient)
    - Differentiable means the slope can be found anywhere.
  - Some activation functions are differentiable and can be used with most optimizer
  - Required for Back-propagation training in Multilayer Perceptron (MLP)
    - Updating of weights requires the slope of the activation function

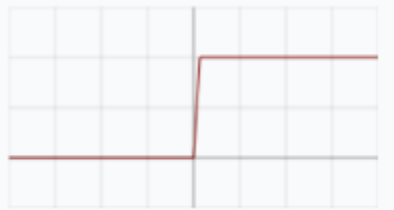
Activation Functions

Linear Function or Identity Function

- Output equals to the input
  - The slope is constant, no calculation needed
- Linear Regression (output is numeric free range)

Heaviside Function or Signed Function

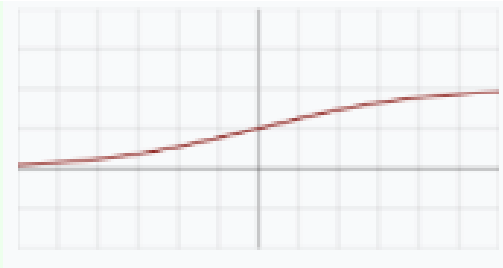
- Output 0 or 1
- Logistic Regression (0 for False and 1 for True or the reverse)
- Not differentiable
  - Not working with back-propagation



Activation Functions

$$\text{Sigmoid } S(x) = \frac{1}{1+e^{-x}}$$

Sigmoid (Logistic) Function



$$f'(x) = f(x)(1 - f(x))$$

- Differentiable everywhere
  - However maximum slope is 0.25
  - <u>Vanishing Gradient Problem!</u>
  - <u>Computational expensive</u> (exponential)
- Output 0 to 1 is useful for output interpreted as probability

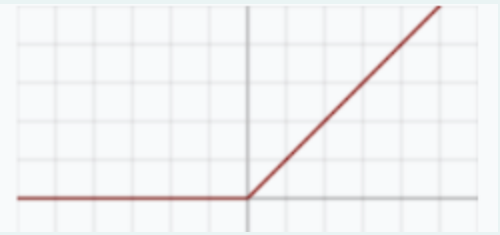
Tanh Function

$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

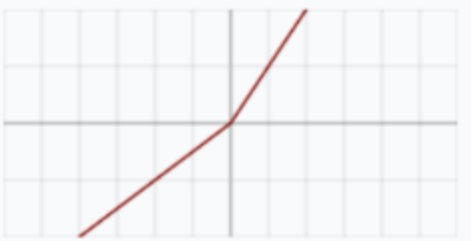
- Used in the first versions of MLP
- Differentiable everywhere
  - <u>Vanishing Gradient Problem!</u>
  - <u>Computational expensive</u> (exponential)
- Useful for classification because the range from -1 to +1
  - Negative cases produce a negative output (clear cut!)

Activation Functions

RELU Function



- REctified Linear Unit
- Computational Cheap
  - Slope is 1 if input > 0
  - Slope is 0 if input < 0
- The slope is sleep for all positive input
  - Useful to provide enough gradient for the gradient descend optimizer
- Became very popular in deep learning (many hidden layers ANN)
- Limitation: negative input are not handled properly
  - All negative values mapped to 0, but some negatives are more negative than other negatives – not differencing the negative cases.
- Leaky RELU
  - To provide some gradient for the negative input



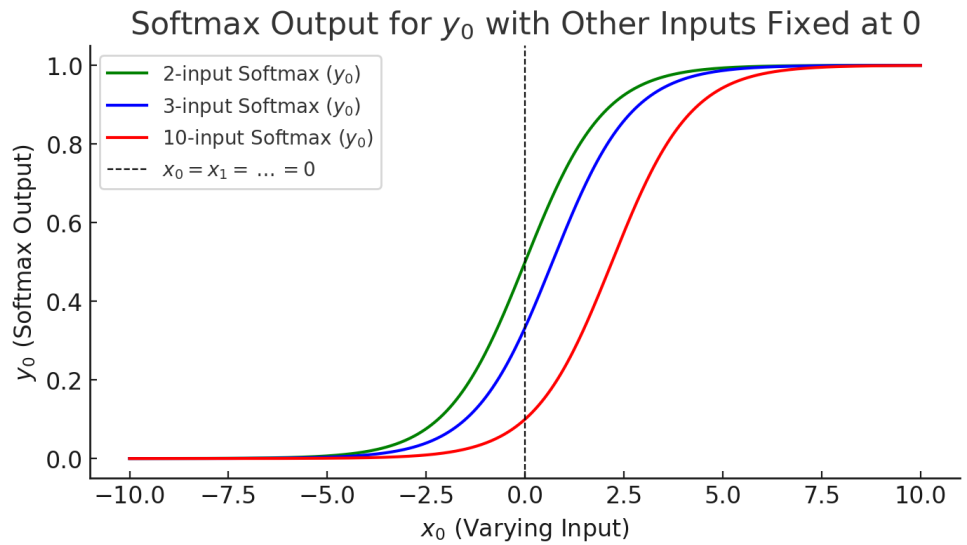
Activation Functions

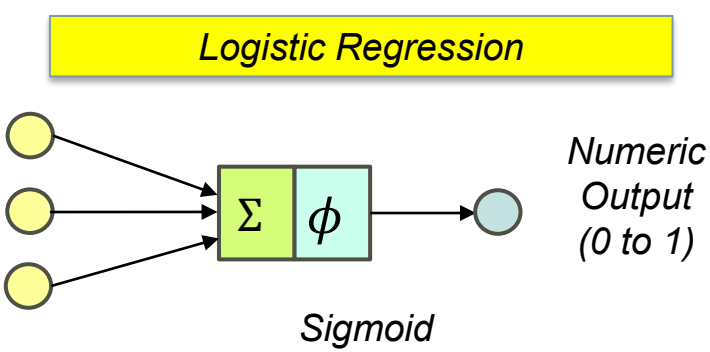
Softmax Function:

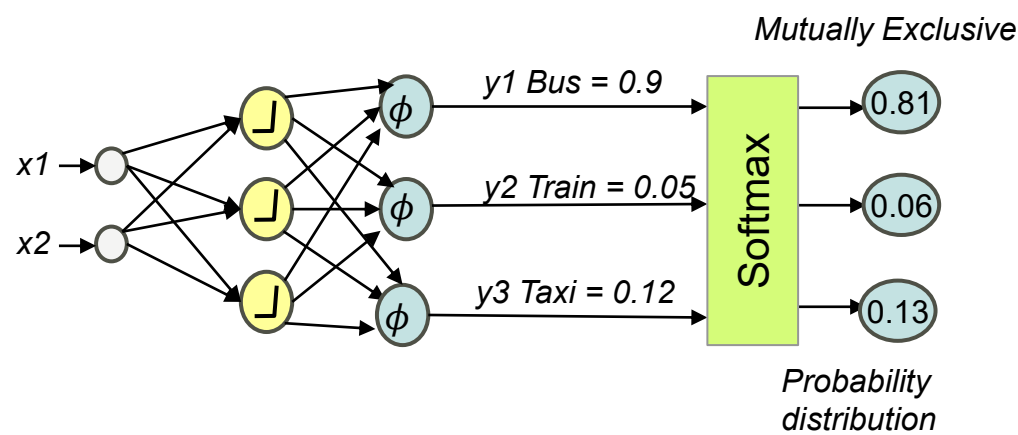
For multi-class classification

$$\text{Softmax } \sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

$$\text{Sigmoid } S(x) = \frac{1}{1+e^{-x}}$$





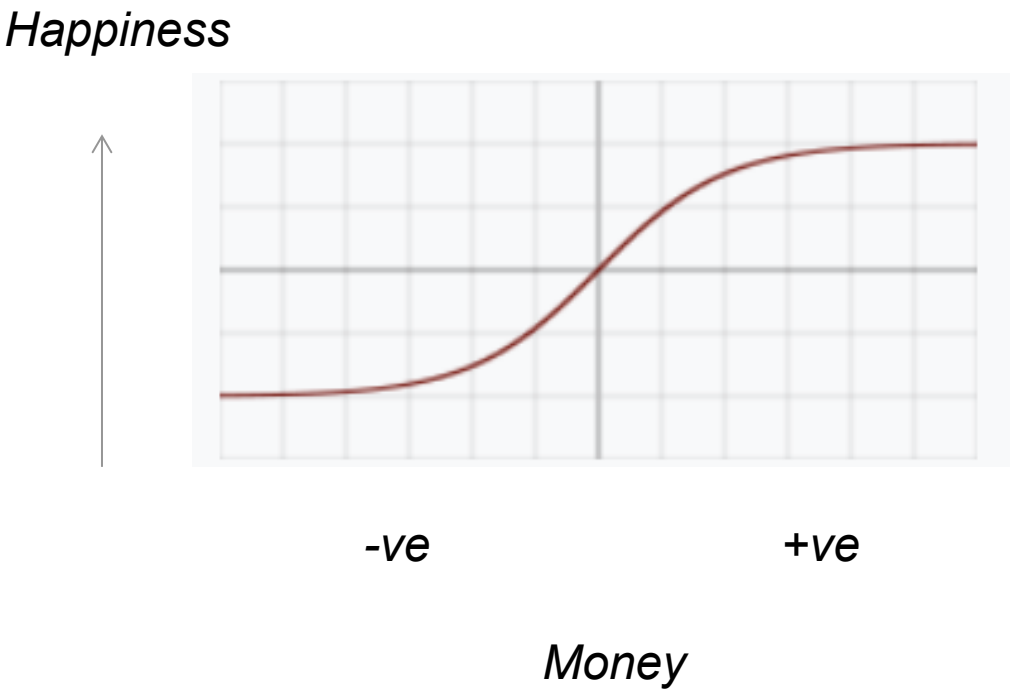


Choice of Activation Functions

- Usually, we choose Relu for hidden layers
- Sigmoid and Tanh have two problems
  - Computation of slope is not simple (though it is not complex but still need to calculate or lookup/interpolate)
  - The Vanishing Gradient Problem
    - The ends are saturated, and the slope is very small
    - In the middle the slope is only 0.25
- The Vanishing Gradient Problem refers to the slow update in each training pass in Backpropagation
  - Backpropagation adjust weights in the input layer by finding the derivative of the network from the output layer through the hidden layer to the input layer
  - The derivatives of each layer are effectively multiplied together
  - More layer means more multiplication
    - Even for maximum slope: 0.25 x 0.25 x 0.25 … becomes very small

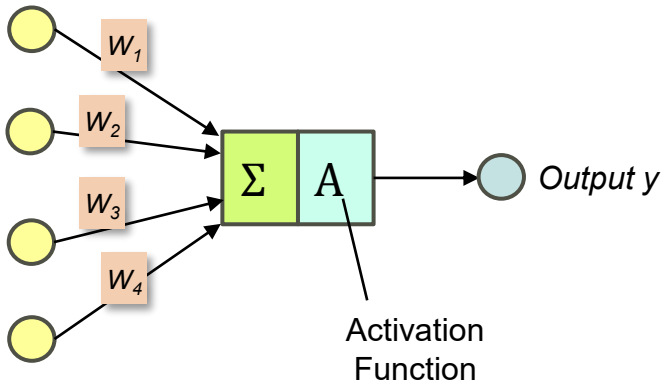
Many Real-World Situations are Non-Linear

- Which curve describes the relation between your money and your happiness?
  - How happier you are if you have just won $10,000?



Non-Linearity of the Activation Function

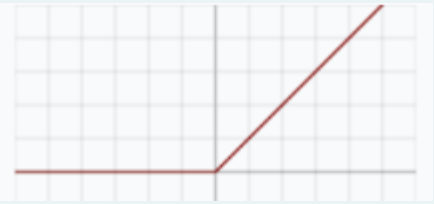
- Many functions and models in the real-world are non-linear
- To model non-linearity, the neural network must include a component that is non-linear
  - The summation part is linear!
- The activation function can be linear or non-linear
  - A linear function is a straight line on a X-Y graph
  - Linear activation functions can only produce linear models
  - Non-linear activation functions are usually used in practice



Common Non-Linear Activation Functions

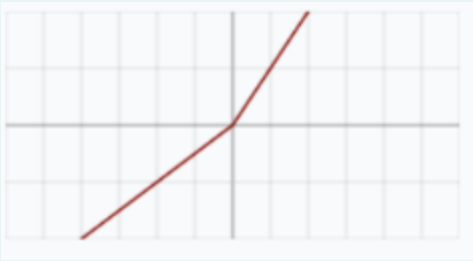
RELU Function

- Slope Calculation Easy
- Output all positive range



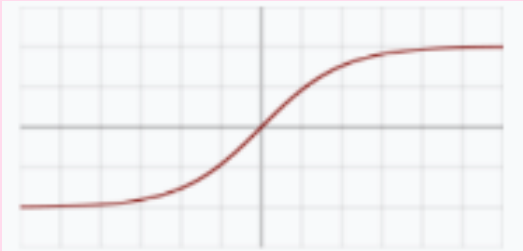
Leaky RELU Function

- Output Full Range



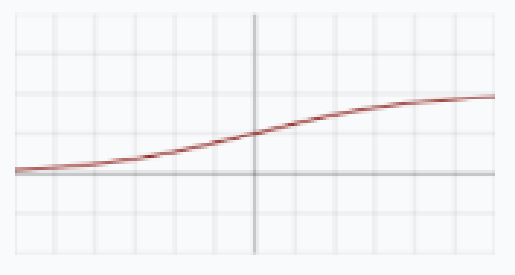
Tanh Function

- Output -1 to 1



Sigmoid (Logistic) Function

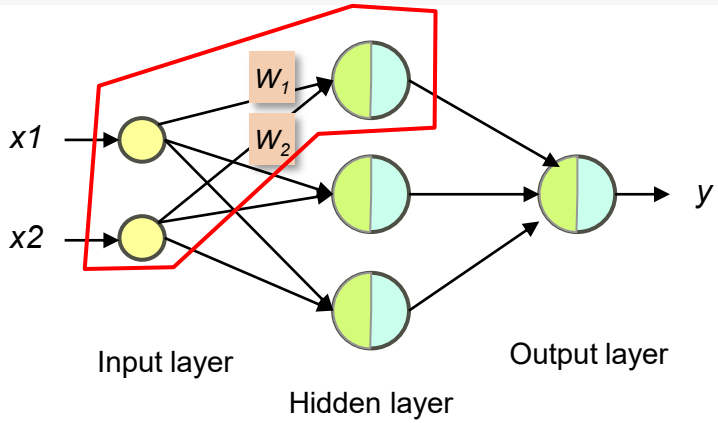
- Output 0 to 1

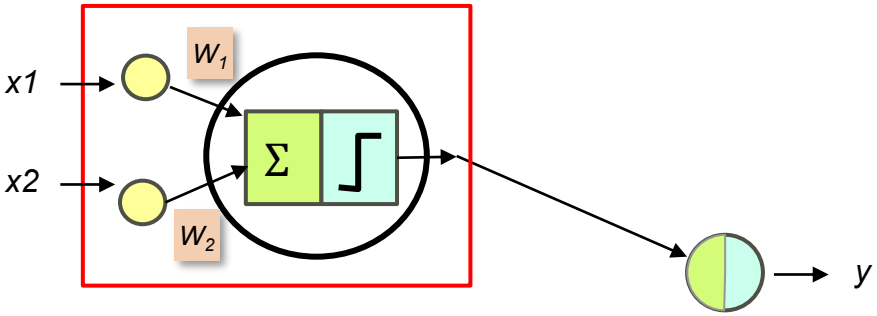


- *Multi-layer Perceptron*

Multi-layer Perceptron (MLP)

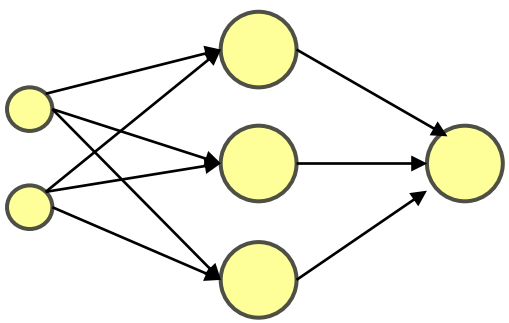
- The input of a perceptron can come from the output of another perceptron
  - Similar, the output of a perceptron can be fed to the input of one or more perceptrons
- MLP consisting of at least 3 fully connected layers: input, hidden (one or more) and output
  - Each layer consists of one or more nodes or perceptrons
  - Fully connection means that the input of a node comes from the output of all nodes in the previous layer, and the output of the node goes to every node in the next layer

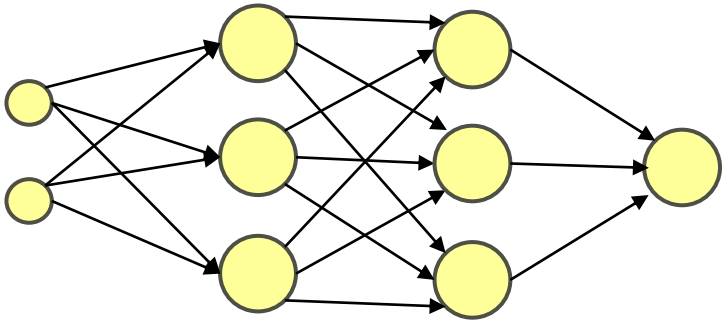




Architectures of Neural Networks: Learning Capacity

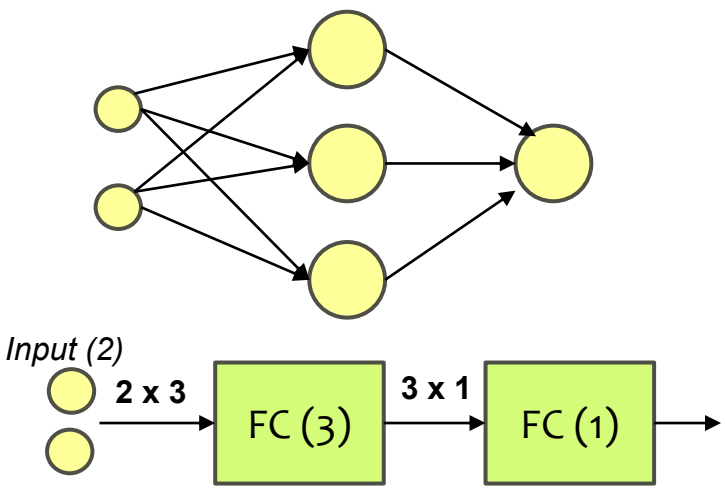
- The number of trainable parameters (weights)
  - Determined by the number of layers, and the number of nodes in each layer
  - Gives ANN models the flexibility to adjust the learning capacity
- More trainable parameters
  - The model able to learn more complex functions and models
  - The training needs more data
  - The training takes more effort (time)
  - The training poses more problems
- Examples: the network on the right is considered a deep neural network
  - The network has two hidden layers



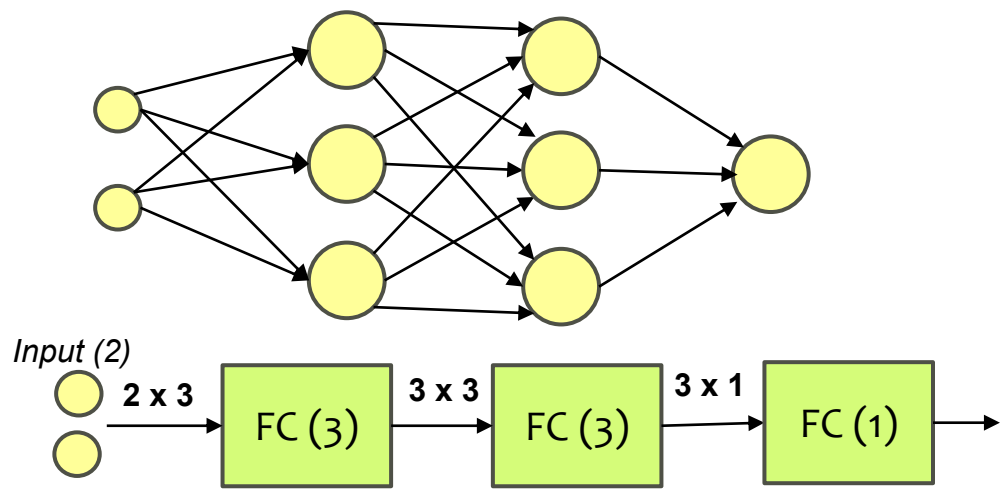


Architectures of Neural Networks: Learning Capacity

- Deep neural networks have two or more hidden layers.
- Diagram representation of ANN
  - A fully connected hidden layer is often referred as FC
  - The interconnections between layers are given
    - The number of trainable parameters (weights)
- Example: number of trainable parameters (weight)
  - Assume no bias neurons



Number of parameters = 2x3+3x1 = 9 weights

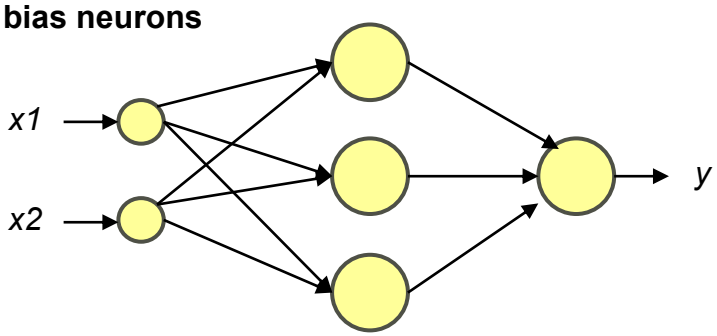


Number of parameters = 2x3+3x3+3x1=18 weights

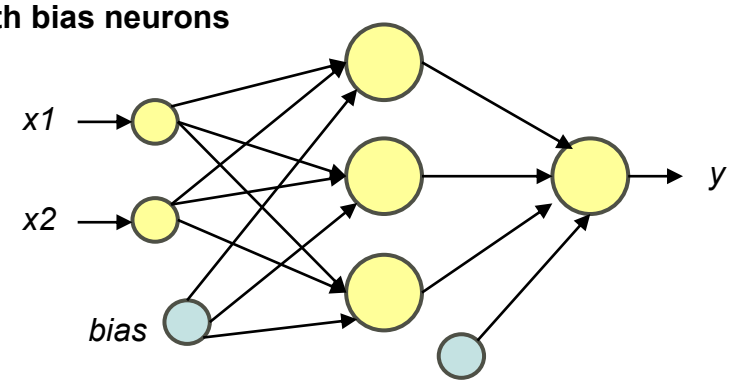
Architectures of Neural Networks: Learning Capacity

- The MLP has a lot more trainable parameters
  - The weights are between two successive layers
    - There are 2 input nodes, 3 nodes in the hidden layer, and 1 output node
  - Assume there is no bias neurons
    - Input-Hidden: 2 x 3 = 6 weights
    - Hidden-Output: 3 x 1 = 3 weights
    - Total = 9 weight parameters
  - Assume bias neurons
    - Input-Hidden: (2+1) x 3 = 9 weights
    - Hidden-Output: (3+1) x 1 = 4 weights
    - Total = 13 weight parameters

No bias neurons

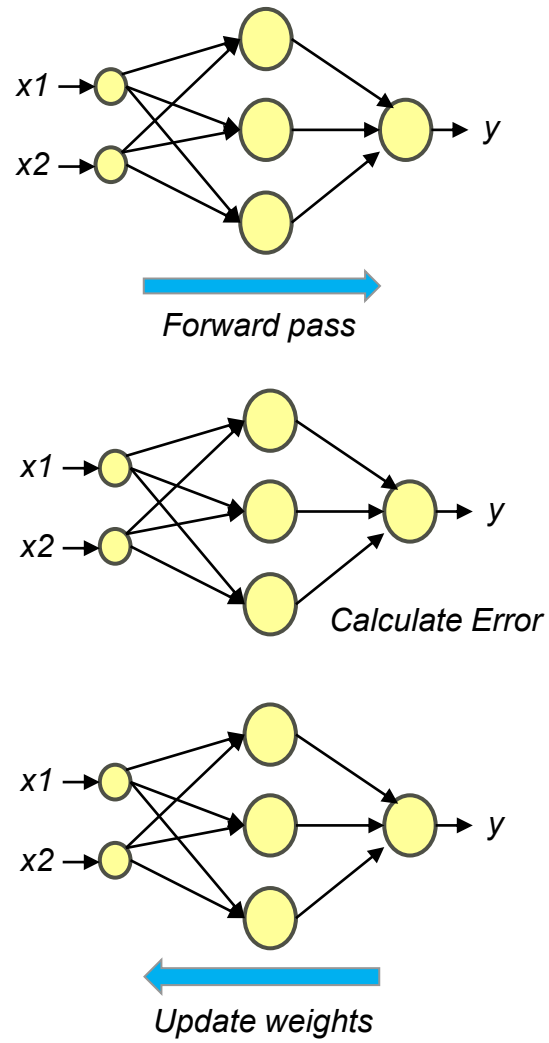


With bias neurons



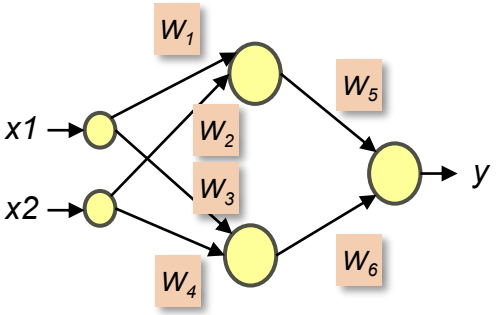
Training MLP:
Back-Propagation and Gradient Descent

- In a single perceptron, the weights are updated according to the error
- In MLP, some weights are before the hidden layer and other weights are between the hidden and output layers
  - An algorithm is needed to update the weights in an effective manner. General steps are listed below.
    - Forward pass: calculate the output of every node of each layer from input to output
    - Calculate error: calculate the error at the output layer (between the model output and the ground-truth output)
    - Weight update: according to an optimizer algorithm, the weights are updated from the output layer back to the input layer

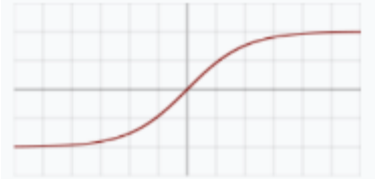


Training MLP:
Back-Propagation and Gradient Descent

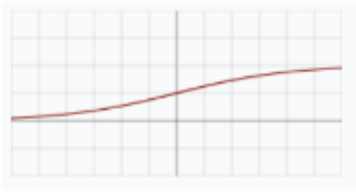
- Backpropagation or backward propagation of errors is a training algorithm
  - It provides formulations for updating the weights based on error
    - The weights are updated one by one, from output to input
  - It adopts gradient descent approach for updating the weight coefficients
  - Requirements on the activation function
    - It must be differentiable so that we can compute the gradient.
    - It should saturate (become constant) at both ends of the range, so that the neuron either fires or does not fire.
    - It should change between the saturation values fairly quickly in the middle.



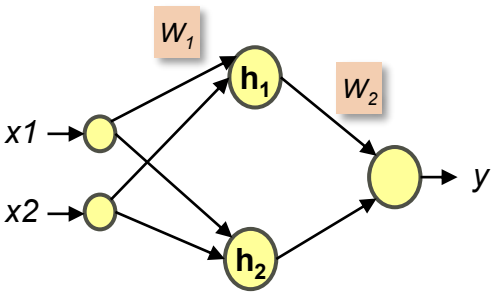
Tanh Function

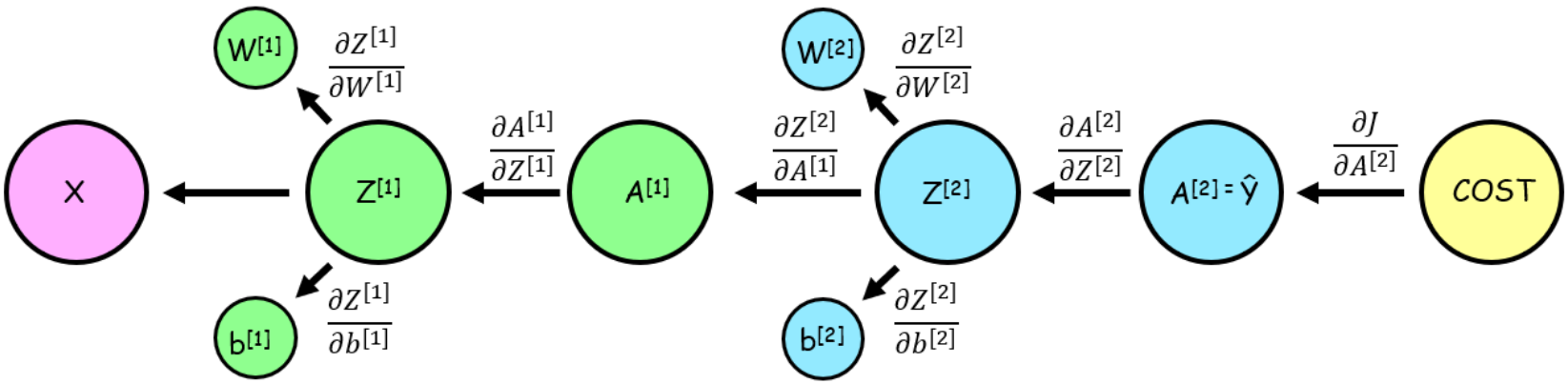


Sigmoid Function

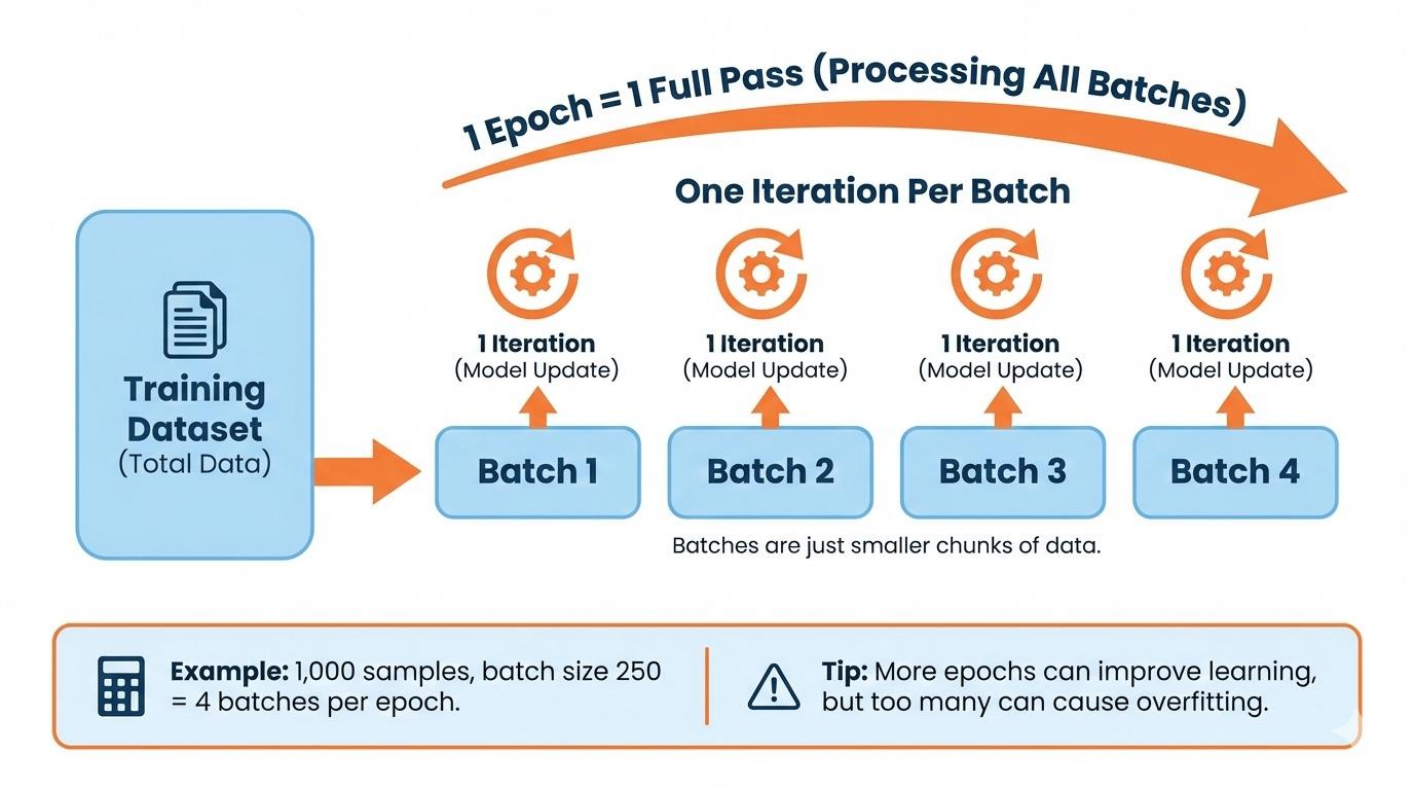


Training MLP:
Back-Propagation and Gradient Descent：Example



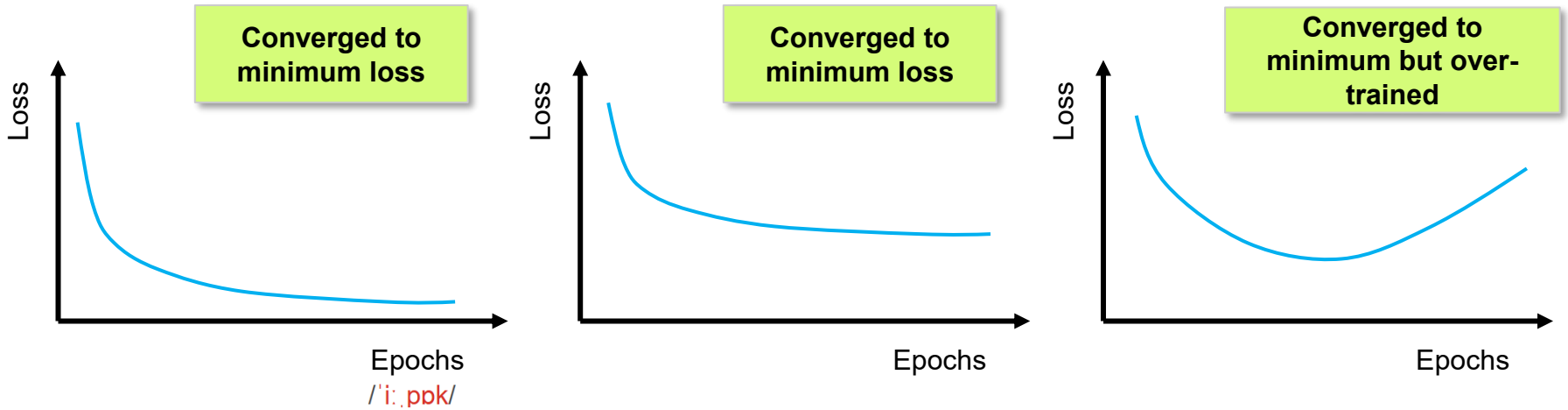


Training MLP:
Epoch and Batch – passing data to the model and update paramaters



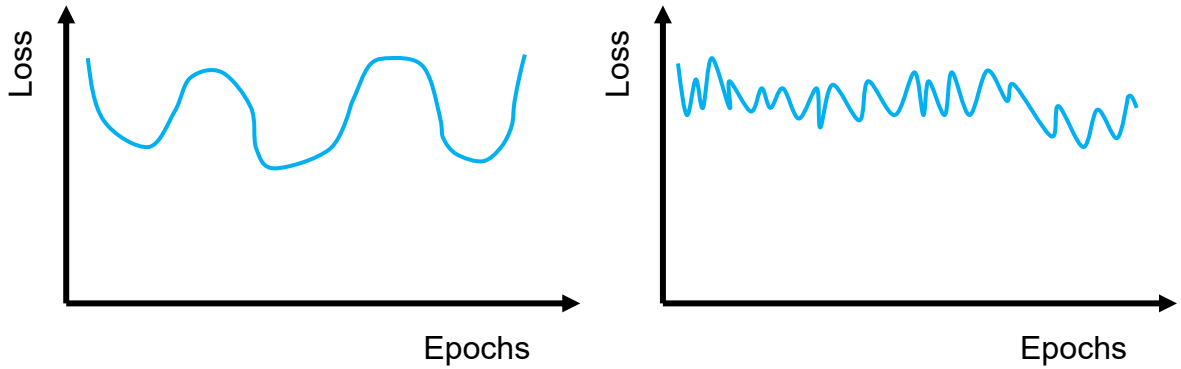
- an **epoch** means one complete pass of the entire training dataset through the model
- A **batch** means a chunk of the dataset used to update the model during an epoch

Training MLP:
Training Error and Convergence



- Convergence in machine learning means that the error is getting close to the minimum
  - The aim of training is for the loss to converge to the minimum

Training MLP:
Failed Convergence



- Sometimes the model training may fail to converge
  - Reasons may include inappropriate training method, non-optimal training parameters, training data inadequacies

Stopping Condition

- The stopping conditions of a training scenario may include
  - A fixed number of epochs
  - A fixed number of epochs with early stopping
    - When the loss is not decreasing much and remaining steady
  - A fixed number of epochs and select the best model
    - Make a copy of every model at each epoch
    - Choose the model that gave the minimum validation error (training error and testing error)

Summary of MLP

- MLPs are useful in research for their ability to solve problems stochastically, which often allows approximate solutions for extremely complex problems like fitness approximation.
- MLPs are universal function approximators, so they can be used to create mathematical models by regression analysis. As classification is a particular case of regression when the response variable is categorical, MLPs make good classifier algorithms.
- MLPs were a popular machine learning solution in the 1980s, finding applications in diverse fields such as speech recognition, image recognition and machine translation software.